

# LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Greedy Exchange Arguments in Practice  
Week 4 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

# Outline

# Why walk through LeetCode problems?

- This week's LEETCODE.md list gives ten Week 4 problems to *practice on your own*. This deck **narrates the thinking process** for two of them, so you can reuse the process on the rest.
- We pick one **Medium** (*Non-overlapping Intervals*) that is almost literally this week's lecture, and one **Hard** (*Course Schedule III*) that extends this week's "job sequencing with deadlines" case study with a heap.

## The four-step process we will repeat twice

- ➊ **First instinct.** Write down the algorithm that follows most directly from re-reading the problem statement – usually brute force or exhaustive search.
- ➋ **Measure why it hurts.** Compute the actual complexity, plug in the problem's constraints, and see the number blow up.
- ➌ **Find the structural insight.** Almost always this is an *exchange argument*: show that a greedy choice can replace any part of an optimal solution without making it worse.
- ➍ **Implement the optimal algorithm**, trace it by hand, then look for the *programming-level* tricks that make the implementation itself correct and fast.

### This mirrors the course

Steps 1–3 are exactly this week's “exchange argument” proof template applied to a new problem; step 4 is what turns a correct algorithm into working code.

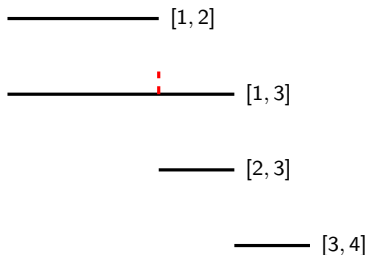
## Problem statement

### LeetCode 435 – Non-overlapping Intervals (Medium)

Given an array of intervals  $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ , return the **minimum number of intervals you need to remove** so that the rest of the intervals are pairwise non-overlapping. Intervals that only touch at an endpoint (e.g.  $[1, 2]$  and  $[2, 3]$ ) do **not** count as overlapping.

- Constraints (LeetCode):  $1 \leq n \leq 10^5$ .
- Removing the fewest intervals to leave a compatible set is the same as **keeping the most** intervals compatible – i.e., this *is* this week's Interval Scheduling problem, with the answer reported as  $n$  minus the optimum.

## Worked example



`intervals = [[1, 2], [1, 3], [2, 3], [3, 4]]`

- $[1, 2]$ ,  $[2, 3]$ ,  $[3, 4]$  are mutually compatible (touching endpoints are fine) – a compatible set of size 3.
- $[1, 3]$  overlaps both  $[1, 2]$  and  $[2, 3]$ , so it cannot join that set.
- **Answer:** remove just  $[1, 3] \Rightarrow \boxed{1}$ .

## First instinct: search over subsets

### Most direct reading of the problem

“Find the fewest removals”  $\Rightarrow$  try every subset of intervals, check which subsets are pairwise compatible, keep the largest compatible subset, and report  $n$  minus its size.

```
1:  $best \leftarrow 0$ 
2: for each subset  $S \subseteq \{1, \dots, n\}$  do
3:   if all pairs in  $S$  are pairwise compatible then
4:      $best \leftarrow \max(best, |S|)$ 
5:   end if
6: end for
7: return  $n - best$ 
```

## Why this is bad

- There are  $2^n$  subsets, and checking pairwise compatibility of a subset of size  $k$  costs  $O(k^2) \Rightarrow$  total time is exponential, roughly  $O(2^n n^2)$  in the worst case.
- At  $n = 10^5$ ,  $2^n$  is astronomically larger than the number of atoms in the observable universe. This does not merely time out – it is not computable in the lifetime of the sun.

### The smell to notice

“Try every subset” is almost never the intended solution to a LeetCode Medium. When the problem is secretly asking for a *maximum compatible subset of intervals*, that is a named problem this course already solved – go find the lecture’s algorithm instead of reinventing search.



# The structural insight

- Minimizing removals  $\equiv$  maximizing the compatible subset kept, and **maximizing a compatible subset of intervals is exactly this week's Interval Scheduling problem.**
- This week's exchange argument already proved: sorting by **finish time** and greedily keeping every interval compatible with everything kept so far produces a maximum compatible set.
- So the answer is simply  $n - |\text{greedy's kept set}|$  – no new proof needed, just a direct application of today's theorem.

## Optimal algorithm: pseudocode

```
1: function MINREMOVALS(intervals)
2:   sort intervals by end time ascending
3:    $prevEnd \leftarrow -\infty$ ;  $kept \leftarrow 0$ 
4:   for  $(s, e)$  in sorted intervals do
5:     if  $s \geq prevEnd$  then
6:        $kept \leftarrow kept + 1$ ;  $prevEnd \leftarrow e$ 
7:     end if
8:   end for
9:   return  $n - kept$ 
10: end function
```

▷ compatible: keep it

Cost:  $O(n \log n)$  for the sort,  $O(n)$  for the scan – this week's exact bound.

## Trace on the worked example

`intervals = [[1,2], [1,3], [2,3], [3,4]]`, sorted by end: `[1,2], [1,3], [2,3], [3,4]` (the two end-3 intervals may appear in either order).

| interval | $s \geq \text{prevEnd?}$ | action                | $\text{prevEnd}$ after |
|----------|--------------------------|-----------------------|------------------------|
| [1, 2]   | $1 \geq -\infty$ , yes   | keep (1st kept)       | 2                      |
| [1, 3]   | $1 \geq 2?$ no           | <b>skip</b> (removed) | 2                      |
| [2, 3]   | $2 \geq 2$ , yes         | keep (2nd kept)       | 3                      |
| [3, 4]   | $3 \geq 3$ , yes         | keep (3rd kept)       | 4                      |

### Answer

$\text{kept} = 3, n = 4 \Rightarrow \text{removals} = 4 - 3 = 1$  – matches the expected output.

## Complexity: naive vs. optimal

| Approach                             | Time                            | Idea                                   |
|--------------------------------------|---------------------------------|----------------------------------------|
| Search over subsets                  | $O(2^n n^2)$                    | try every compatible subset            |
| <b>Sort by end time, greedy scan</b> | <b><math>O(n \log n)</math></b> | this week's earliest-finish-time-first |

# Python implementation

```
1 def eraseOverlapIntervals(intervals):
2     intervals.sort(key=lambda iv: iv[1])    # sort by END time, not start time!
3     prev_end = float('-inf')
4     kept = 0
5     for start, end in intervals:
6         if start >= prev_end:                # compatible with everything kept so
7             far
8             kept += 1
9             prev_end = end
10    return len(intervals) - kept
```

## Implementation tips & tricks (the programming side)

- **Sort by end time, not start time.** The single most common bug on this problem: sorting by `iv[0]` (start) instead of `iv[1]` (end). The exchange-argument proof only holds for the finish-time ordering – swap the key and the algorithm silently returns a wrong (usually too-small) kept count on inputs where it matters.
- **The comparison must be  $\geq$ , not  $>$ .** Touching endpoints ( $[1, 2]$  then  $[2, 3]$ ) are compatible by this problem's definition. Using strict  $>$  would incorrectly reject touching intervals – an off-by-one that only shows up on boundary test cases.
- **Track only the last kept end time, not the whole kept set.** Because intervals are processed in sorted order, one running scalar (`prev_end`) is enough – no need to store or re-scan the kept intervals, which is what keeps this  $O(n)$  after the sort instead of  $O(n^2)$ .

## Problem statement

### LeetCode 630 – Course Schedule III (**Hard**)

You are given `courses[i] = [duration_i, lastDay_i]`: course  $i$  takes `duration_i` consecutive days and must be **finished** on or before day `lastDay_i`. You take courses one at a time (no overlap), starting on day 1. Return the **maximum number of courses** you can take.

- Constraints (LeetCode):  $1 \leq n \leq 10^4$ .
- This is exactly this week's **job sequencing with deadlines** case study, generalized so each job's duration need not be 1 – and now the question is “how many,” not “how much profit.”

## Worked example

`courses = [[100,200] , [1000,1250] , [200,1300] , [2000,3200]]`

- Sorted by deadline: (100, 200), (1000, 1250), (200, 1300), (2000, 3200).
- Taking the first three in that order costs  $100 + 1000 + 200 = 1300$  days total, which fits before the tightest of their three deadlines (1300).
- The fourth course alone needs 2000 more days on top of that – pushing the total to  $3300 > 3200$ , its own deadline. It cannot be added without dropping something.
- **Answer:** 3 courses (drop the 2000-day course).



## First instinct: try every ordering

### Most direct reading of the problem

“Which subset, taken in which order, fits?”  $\Rightarrow$  try every permutation of every subset of courses, check feasibility (running total of durations never exceeds the deadline of the course just placed), keep the largest feasible subset.

```
1:  $best \leftarrow 0$ 
2: for each subset  $S$ , each ordering  $\pi$  of  $S$  do
3:   if  $\pi$  is feasible (cumulative duration  $\leq$  each course's deadline in order) then
4:      $best \leftarrow \max(best, |S|)$ 
5:   end if
6: end for
7: return  $best$ 
```

## Why this is bad

- There are up to  $2^n$  subsets, each with up to  $n!$  orderings to check.
- At  $n = 10^4$ , this is not merely exponential – it is *super-exponential*, hopelessly beyond any brute-force search.

### The smell to notice (again)

“Try every ordering” ignores a fact this week’s lecture already proved for a close cousin of this problem: when jobs have deadlines, a fixed, simple ordering rule (sort by deadline) is enough – you never need to consider arbitrary orderings.

# The structural insight

- **Always take courses in increasing order of deadline.** Reordering two selected courses so the earlier-deadline one goes first can only help feasibility (an exchange argument identical in spirit to this week's minimizing-lateness proof) – so an optimal solution can always be rearranged into deadline order without losing any courses.
- Once the order is fixed, scan courses by deadline and greedily try to take each one. If taking it would blow the running total past *its own* deadline (the tightest deadline seen so far, since we're in deadline order), the schedule is over-subscribed – and the best fix is to **drop whichever selected course has the longest duration**: that frees the most time while only affecting a course whose deadline is at least as loose.
- This is a second exchange argument: swapping the longest-duration course out for a shorter one never increases total time and never decreases the count of courses taken.

# Optimal algorithm: pseudocode

```
1: function MAXCOURSES(courses)
2:   sort courses by lastDay ascending
3:    $H \leftarrow$  empty max-heap (keyed by duration);  $total \leftarrow 0$ 
4:   for (duration, lastDay) in sorted courses do
5:      $total \leftarrow total + duration$ ; push duration onto  $H$ 
6:     if  $total > lastDay$  then
7:        $total \leftarrow total - \text{POPMAX}(H)$ 
8:     end if
9:   end for
10:  return  $|H|$ 
11: end function
```

▷ drop the longest course so far

Cost:  $O(n \log n)$  – one heap push/pop per course.

## Trace on the worked example

`courses` = `[[100,200],[1000,1250],[200,1300],[2000,3200]]`, already sorted by deadline.

| course ( $d, \ell$ ) | <i>total</i> after push | $> \ell$ ?                   | action                                         |
|----------------------|-------------------------|------------------------------|------------------------------------------------|
| (100, 200)           | 100                     | no                           | keep – $H = \{100\}$                           |
| (1000, 1250)         | 1100                    | no                           | keep – $H = \{100, 1000\}$                     |
| (200, 1300)          | 1300                    | no ( $1300 \leq 1300$ )      | keep – $H = \{100, 1000, 200\}$                |
| (2000, 3200)         | 3300                    | <b>yes</b> ( $3300 > 3200$ ) | pop max (2000), <i>total</i> $\leftarrow$ 1300 |

### Answer

$|H| = 3$  at the end – the 2000-day course is popped right back off the heap it was just pushed onto, since it is (unsurprisingly) the longest course seen.

## Complexity: naive vs. optimal

| Approach                           | Time                            | Idea                      |
|------------------------------------|---------------------------------|---------------------------|
| Subsets $\times$ orderings         | super-exponential               | $2^n \cdot n!$ candidates |
| <b>Sort by deadline + max-heap</b> | <b><math>O(n \log n)</math></b> | two exchange arguments    |

# Python implementation

```
1 import heapq
2
3 def scheduleCourse(courses):
4     courses.sort(key=lambda c: c[1])    # sort by deadline (lastDay)
5     max_heap = []                      # Python heapq is a MIN-heap...
6     total = 0
7     for duration, last_day in courses:
8         total += duration
9         heapq.heappush(max_heap, -duration)    # ...so push -duration to simulate
            max
10        if total > last_day:
11            total += heapq.heappop(max_heap)    # pop the (negated) longest duration
12    return len(max_heap)
```

## Implementation tips & tricks (the programming side)

- **Python's `heapq` is a min-heap only.** The algorithm needs a *max*-heap of durations. The standard trick is to push the **negated** duration (`-duration`); the smallest negated value is the largest true value. Popping then gives `-heapq.heappop(...)` back, or – as above – simply `total += heapq.heappop(max_heap)` directly, since adding a negative number is the same as subtracting its absolute value.
- **Sort key must be `lastDay` (index 1), not `duration`.** Sorting by duration instead silently breaks the exchange-argument guarantee – the algorithm will still run and return a number, just not always the right one, which makes this bug easy to miss on small test cases.
- `len(max_heap)` **is the answer, not a separate counter.** Because every kept course corresponds to exactly one surviving heap entry, the heap's size after the scan already *is* the count – no need to maintain a redundant counter that could drift out of sync with the heap.



## Summary: two problems, one lesson

- 1 **Non-overlapping Intervals** is not a new problem in disguise – it *is* this week's Interval Scheduling problem, with the answer read off as  $n$  minus the greedy's kept count. Recognizing the disguise is the whole difficulty.
- 2 **Course Schedule III** extends this week's job-sequencing-with-deadlines case study: sort by deadline (exchange argument #1), then use a second exchange argument (always evict the longest-duration course when over budget) to keep the schedule optimal with a single max-heap.
- 3 Both times, the naive instinct was combinatorial search (subsets, or subsets-times-orderings); both times, an exchange argument identical in spirit to today's lecture collapsed the search down to one sort and one linear/heap-based scan.
- 4 The implementation tricks (sort key choice,  $\geq$  vs.  $>$ , simulating a max-heap with negation) are a second, independent layer: getting the proof right is necessary but not sufficient – the code has to match the proof exactly.

### Keep practicing

LEETCODE.md lists eight more Week 4 problems (*Merge Intervals*, *Meeting Rooms II*, *Gas Station*, *Task Scheduler*, ...) – try applying the same four-step process to each of them.